LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
SEMESTER III



TUGAS PRAKTIKUM ACARA 22

NAMA:

RAJATA PRINGGIT BINTANG BIHAR

GOLONGAN:

A

NIM:

E41250264

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada Acara 21, dokumentasi API dibuat menggunakan Markdown (API_DOCUMENTATION.md) dan Postman. Meski mudah dibaca, Markdown belum memiliki struktur baku sehingga sulit divalidasi atau diolah mesin secara otomatis.
Untuk mengatasinya, Acara 22 mengonversi dokumentasi tersebut ke format OpenAPI Specification 3.0 (openapi.yaml). Path yang digunakan menyesuaikan peta Router (bukan nama file .php), lalu divisualisasikan di Swagger Editor agar menjadi dokumentasi yang interaktif.

2. Tujuan

1. Membuat berkas docs/openapi.yaml berisi kerangka OpenAPI 3.0 (openapi, info, servers, paths).

2. Mendefinisikan path /login dan /mahasiswa mengikuti peta Router.

3. Melengkapi path PUT dan DELETE pada /mahasiswa serta path /register (tugas mandiri 1).

4. Menambahkan komponen securitySchemes bertipe http/bearer pada bagian components (tugas mandiri 2).

5. Memvisualisasikan dokumen pada Swagger Editor dan menyimpan tangkapan layarnya sebagai lampiran di docs/.

6. Memvalidasi dokumen terhadap spesifikasi OpenAPI.

3. Dasar Teori

OpenAPI Specification (dahulu Swagger Specification) adalah format standar industri untuk mendeskripsikan REST API dalam dokumen YAML atau JSON yang dapat dibaca manusia dan mesin. Spesifikasi Swagger 2.0 didonasikan kepada OpenAPI Initiative di bawah Linux Foundation pada tahun 2015, dan OpenAPI 3.x membawa berbagai penyempurnaan. Dokumen OpenAPI tersusun atas beberapa bagian utama (Tabel 1).

Tabel 1. Bagian utama dokumen OpenAPI 3.0

Bagian	**Fungsi**
openapi	Versi spesifikasi yang dipakai (3.0.0).
info	Judul, versi, dan deskripsi API.
servers	Alamat dasar (base URL) tempat API berjalan.
paths	Daftar path beserta method, parameter, requestBody, dan responses.
components	Skema, response, dan securitySchemes yang dapat dipakai ulang lewat $ref.
security	Skema autentikasi yang berlaku pada operasi (di tingkat dokumen atau operasi).

Swagger Editor adalah alat berbasis web untuk menulis dokumen OpenAPI dengan pratinjau langsung, sedangkan Swagger UI menghasilkan halaman dokumentasi interaktif. Dibandingkan Markdown, OpenAPI memiliki struktur baku, dapat divalidasi otomatis, serta dapat dipakai untuk menghasilkan dokumentasi interaktif maupun kode klien.

HASIL PRAKTIK

1. Implementasi

a. Perubahan File

Acara 22 tidak mengubah kode program. Yang ditambahkan hanya dokumentasi OpenAPI; dokumentasi Markdown dari Acara 21 tetap dipertahankan.

Tabel 2. Daftar perubahan file

File	Status	Keterangan
docs/openapi.yaml	Baru	Dokumentasi OpenAPI 3.0
docs/swagger-preview.png	Baru	Lampiran tangkapan layar Swagger Editor
postman/Local.postman_environment.json	Diubah	base_url mengarah ke folder si-akademik_acara22
README.md	Diubah	Ringkasan perubahan Acara 22

b. Langkah 1 - Membuat Berkas openapi.yaml

Berkas docs/openapi.yaml dibuat berisi kerangka awal: versi spesifikasi, informasi API, alamat server, dan kumpulan path. Alamat server mengikuti folder proyek pada XAMPP dan berakhir di public/ karena Front Controller berada di sana.

```
openapi: 3.0.0
info:
  title: API Sistem Informasi Akademik (MVC)
  version: 1.0.0
  description: 'Dokumentasi OpenAPI untuk API Sistem Informasi Akademik (PHP
Native, arsitektur MVC).
    Path mengikuti resource pada peta Router, bukan nama berkas .php. Endpoint
yang bertanda gembok memerlukan
    header `Authorization: Bearer <token>`; token diperoleh dari `POST /login`
dan berlaku 5 menit.'
servers:
- url: http://localhost/setoran_BKPM-1/si-akademik_acara22/public
  description: Server lokal (XAMPP)
security: []
tags:
- name: Autentikasi
  description: AuthController
- name: Mahasiswa
  description: MahasiswaController
- name: Buku
  description: BukuController (tambahan latihan, belum diproteksi token)
paths:
```

c. Langkah 2 - Mendefinisikan Path /login

Path mengikuti resource pada peta Router. POST /login menerima body JSON berisi username dan password, dan dijawab 200 (berhasil), 400 (field kosong), atau 401 (kredensial tidak valid). Skema request dan response tidak ditulis berulang, melainkan dirujuk dari components/schemas dengan $ref. Contoh response diambil dari hasil pengujian nyata pada Acara 19 sampai 21. Berikut cuplikannya.

```
  /login:
    post:
      operationId: login
      tags:
      - Autentikasi
      summary: Autentikasi pengguna (AuthController::login)
      description: Memverifikasi kata sandi dengan `password_verify` dan
menerbitkan token JWT (HS256)
        yang berlaku 300 detik. Method selain POST dijawab 405.
      security: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/LoginInput'
            example:
              username: budi
              password: rahasia123
      responses:
        '200':
          description: Login berhasil
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/LoginResponse'
              examples:
                sukses:
                  summary: Login berhasil
                  value:
                    success: true
                    message: Login berhasil
                    data:
                      token: eyJhbGciOiJIUzI1NiIs...<token JWT>
                      role: user
                    timestamp: '2026-10-21T10:00:00+07:00'
      ...
```

d. Langkah 3 - Mendefinisikan Path /mahasiswa

Path /mahasiswa memuat method GET (publik, dengan parameter query id opsional) dan POST (memerlukan token, ditandai security: bearerAuth). Judul setiap operasi memuat nama Controller dan method penanganannya.

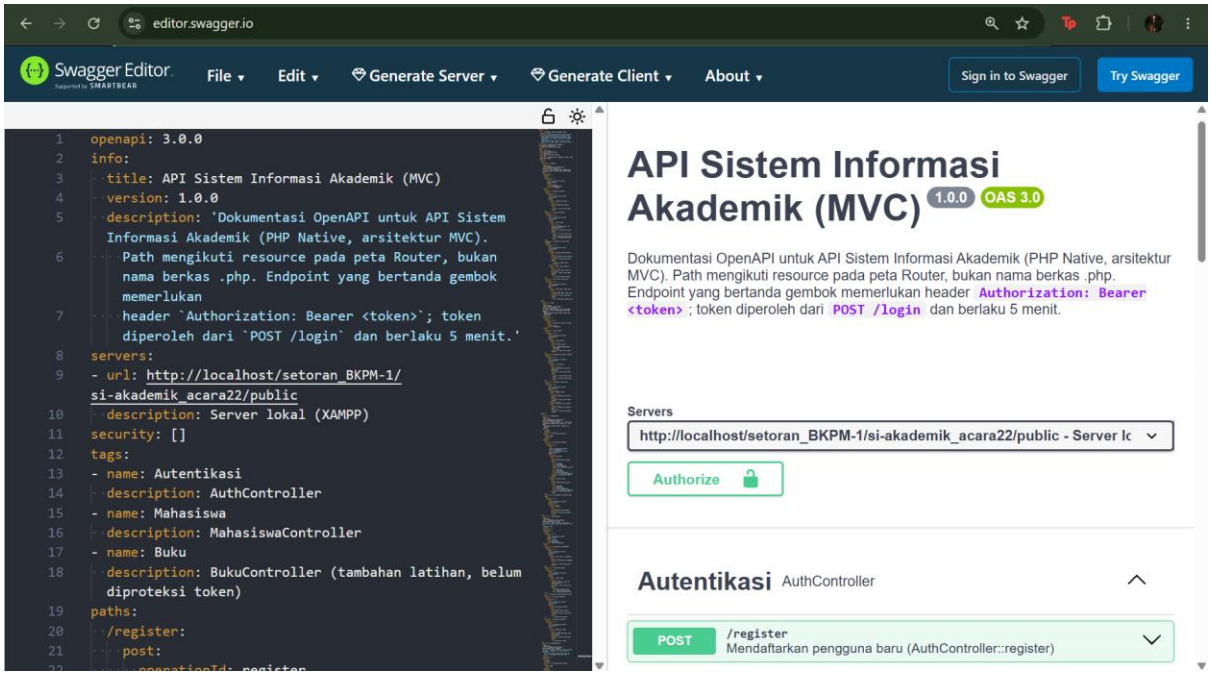
```
  /mahasiswa:
    get:
      operationId: getMahasiswa
      tags:
      - Mahasiswa
      summary: Mengambil data mahasiswa (MahasiswaController::index)
      description: Tanpa parameter `id`, seluruh data dikembalikan sebagai
array. Dengan `id`, satu data
        dikembalikan sebagai objek. Endpoint ini publik.
      parameters:
      - name: id
        in: query
        required: false
        description: Id mahasiswa (opsional)
        schema:
          type: integer
          minimum: 1
        example: 1
      responses:
        '200':
          description: Data berhasil diambil
          content:
            application/json:
              schema:
                oneOf:
                - $ref: '#/components/schemas/MahasiswaListResponse'
                - $ref: '#/components/schemas/MahasiswaResponse'
              examples:
                semua-data:
                  summary: Semua data
                  value:
                    success: true
                    message: Data berhasil diambil
                    data:
                    - id: '2'
                      nim: '220100002'
                      nama: Siti Aminah
                      prodi: Sistem Informasi
                      created_at: '2026-10-21 10:00:00'
                    - id: '1'
                      nim: '220100001'
                      nama: Budi Santoso
                      prodi: Teknik Informatika
```

e. Langkah 4 - Memvisualisasikan Dokumentasi

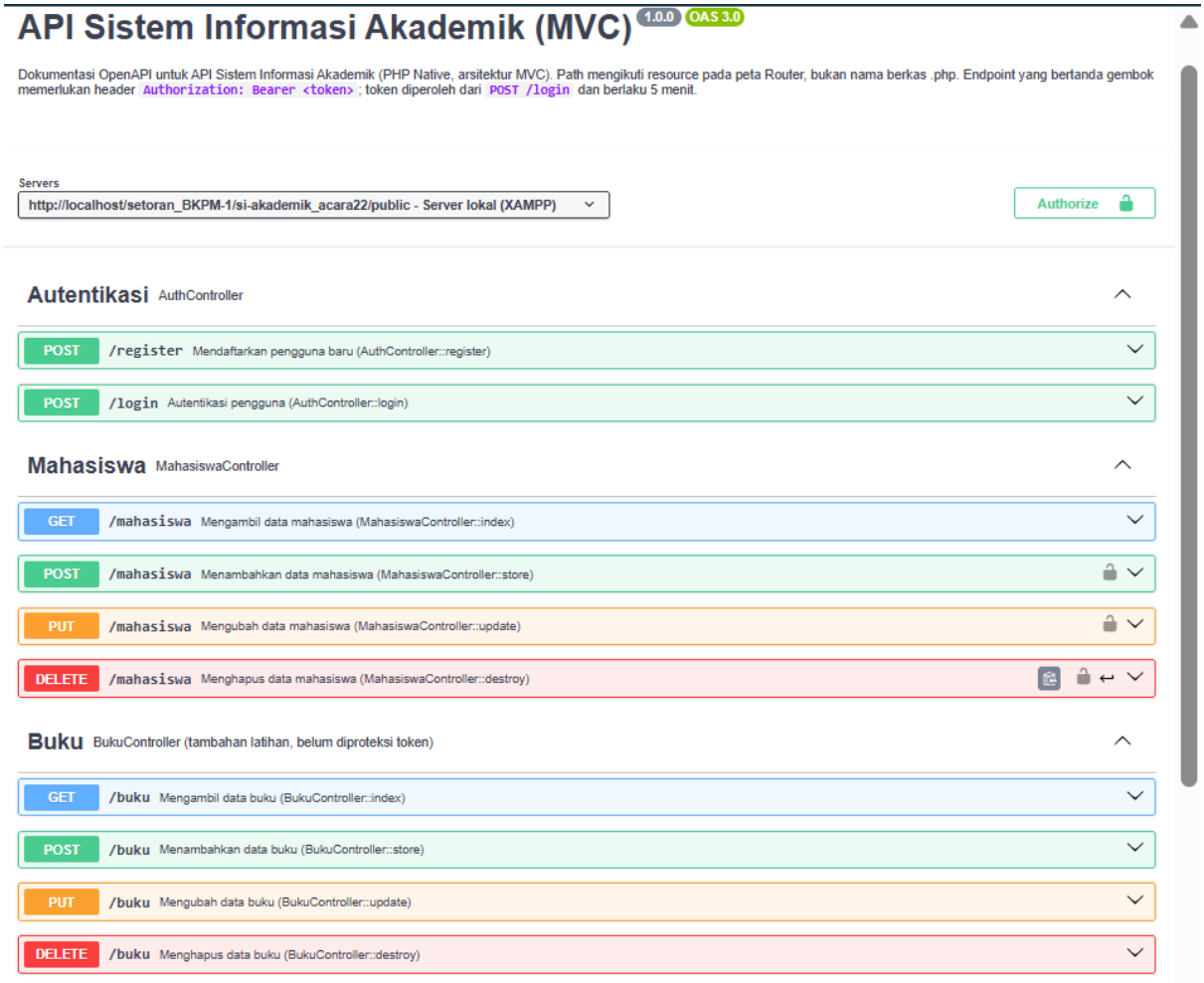
1. Buka https://editor.swagger.io pada browser.

2. Tempelkan seluruh isi openapi.yaml pada panel kiri editor, lalu amati pratinjau pada panel kanan (Gambar 1 dan 2).

3. Simpan tangkapan layar hasil visualisasi sebagai lampiran dokumentasi proyek pada docs/ (Gambar 5).



Gambar 1. Swagger Editor dengan isi openapi.yaml



Gambar 2. Pratinjau daftar endpoint pada Swagger UI

2. Tugas Praktik Mandiri

a. Melengkapi Path PUT, DELETE /mahasiswa dan /register

Path dilengkapi mengikuti pola yang sama. PUT dan DELETE pada /mahasiswa memakai parameter query id yang wajib dan memerlukan token, sedangkan POST /register bersifat publik (security: [] meniadakan keamanan di tingkat dokumen). Cuplikan keterangan perilaku pada deskripsi PUT dan DELETE mencatat bahwa keberadaan id tidak diperiksa oleh kode.

```
    put:
      operationId: updateMahasiswa
      tags:
      - Mahasiswa
      summary: Mengubah data mahasiswa (MahasiswaController::update)
      description: 'Semua field harus dikirim lengkap. Perhatian: keberadaan
`id` tidak diperiksa sehingga
        `id` yang tidak ada tetap dijawab 200, dan body `{}` menghasilkan 500
tanpa JSON.'
      security:
      - bearerAuth: []
      parameters:
      - name: id
```

```yaml
      in: query
      required: true
      description: Id mahasiswa yang diubah
      schema:
        type: integer
        minimum: 1
      example: 1
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/MahasiswaInput'
          example:
            nim: '220100001'
            nama: Budi Santoso Putra
            prodi: Teknik Informatika
    responses:
      '200':
        description: Data berhasil diperbarui
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ApiResponse'
            examples:
              sukses:
                summary: Data berhasil diperbarui
                value:
                  success: true
                  message: Data berhasil diperbarui
                  data: null
                  timestamp: '2026-10-21T10:00:00+07:00'
      '401':
        $ref: '#/components/responses/Unauthorized'
      '409':
        description: NIM milik data lain
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/ErrorResponse'
            examples:
              duplikat:
                summary: NIM milik data lain
                value:
                  success: false
                  message: NIM sudah terdaftar
                  data: null
                  timestamp: '2026-10-21T10:00:00+07:00'
```

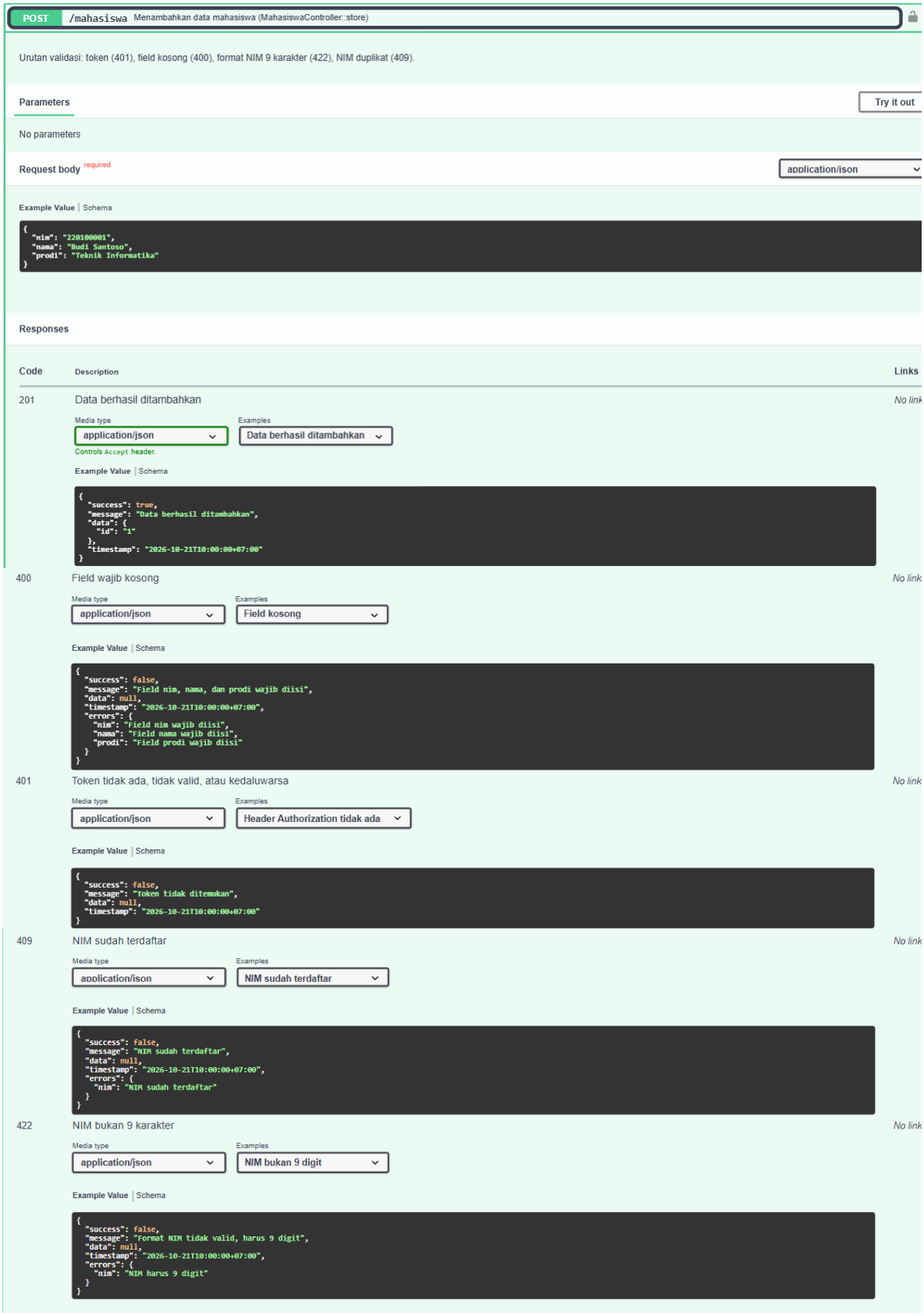
```
                    errors:
                      nim: NIM sudah terdaftar
        '422':
          description: NIM bukan 9 karakter
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ErrorResponse'
              examples:
                nim:
                  summary: NIM bukan 9 digit
                  value:
                    success: false
                    message: Format NIM tidak valid, harus 9 digit
                    data: null
                    timestamp: '2026-10-21T10:00:00+07:00'
                    errors:
                      nim: NIM harus 9 digit
    delete:
      operationId: deleteMahasiswa
      tags:
      - Mahasiswa
      summary: Menghapus data mahasiswa (MahasiswaController::destroy)
      description: 'Perhatian: keberadaan `id` tidak diperiksa sehingga `id`
yang tidak ada tetap dijawab
        200.'
      security:
      - bearerAuth: []
      parameters:
      - name: id
        in: query
        required: true
        description: Id mahasiswa yang dihapus
        schema:
          type: integer
          minimum: 1
        example: 1
...
```

```
  /register:
    post:
      operationId: register
      tags:
      - Autentikasi
      summary: Mendaftarkan pengguna baru (AuthController::register)
      description: Kata sandi disimpan sebagai hash (`password_hash`). Method
selain POST dijawab 405.
      security: []
```

```
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/RegisterInput'
            example:
              username: budi
              email: budi@example.com
              password: rahasia123
...
```

Tabel 3. Ringkasan path, method, dan kode status pada openapi.yaml

Path	Method	operationId	Keamanan	Kode status
/register	POST	register	Publik	201, 400, 409, 422
/login	POST	login	Publik	200, 400, 401
/mahasiswa	GET	getMahasiswa	Publik	200, 404
/mahasiswa	POST	createMahasiswa	bearerAuth	201, 400, 401, 409, 422
/mahasiswa	PUT	updateMahasiswa	bearerAuth	200, 401, 409, 422
/mahasiswa	DELETE	deleteMahasiswa	bearerAuth	200, 401
/buku	GET	getBuku	Publik	200, 404
/buku	POST	createBuku	Publik	201, 400
/buku	PUT	updateBuku	Publik	200
/buku	DELETE	deleteBuku	Publik	200

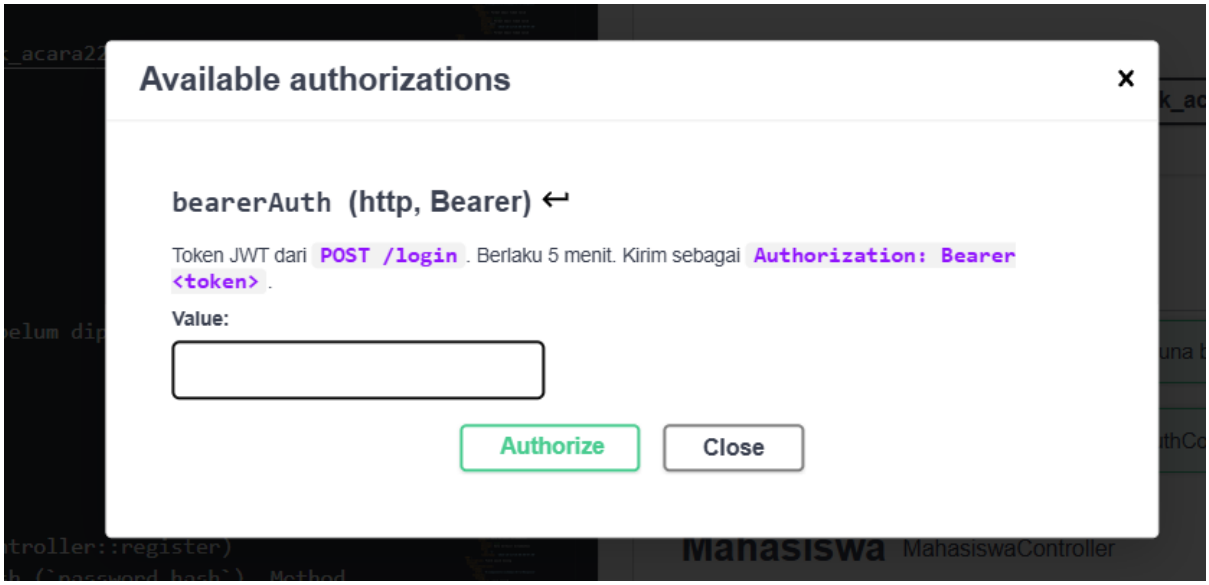


Gambar 3. Operasi /mahasiswa yang memerlukan token ditandai ikon gembok

b. Komponen securitySchemes

Skema keamanan bearerAuth bertipe http dengan scheme bearer dan bearerFormat JWT ditambahkan pada components. Operasi yang dilindungi merujuknya melalui security: [{ bearerAuth: [] }]. Pada Swagger UI muncul tombol Authorize untuk memasukkan token (Gambar 4).

```
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
      description: 'Token JWT dari `POST /login`. Berlaku 5 menit. Kirim
sebagai `Authorization: Bearer
        <token>`.'
```



Gambar 4. Dialog Authorize pada Swagger UI

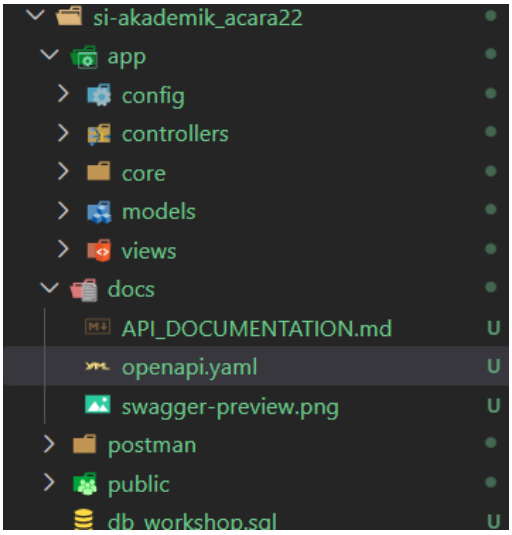
3. Validasi Dokumen

Selain pratinjau, dokumen divalidasi dengan dua validator (Tabel 4). Validasi pertama memastikan dokumen sesuai spesifikasi OpenAPI 3.0; validasi kedua adalah linter yang juga memeriksa praktik penulisan.

Tabel 4. Hasil validasi openapi.yaml

Alat	**Hasil**	**Keterangan**
openapi-spec-validator 0.9.0	Valid	Sesuai OpenAPI 3.0.0
Redocly CLI 1.25.0	Valid	0 error, 4 peringatan non-kritis

Keempat peringatan Redocly bersifat non-kritis: alamat server menunjuk ke localhost, tidak ada field license pada info, serta PUT dan DELETE /buku tidak memiliki response 4xx karena kodenya memang belum memvalidasi atau memeriksa data.



Gambar 5. Lampiran tangkapan layar pada folder docs/

4. Pembahasan

Konversi ke OpenAPI memberi tiga keuntungan dibanding Markdown. Pertama, strukturnya baku sehingga dapat divalidasi: kesalahan penulisan terdeteksi oleh validator, bukan oleh pembaca. Kedua, skema request dan response didefinisikan sekali pada components dan dirujuk dengan $ref sehingga tidak ada pengulangan. Ketiga, dokumen dapat divisualisasikan sebagai halaman interaktif tanpa menulis tampilan sendiri. Contoh response pada dokumen diambil dari pengujian nyata, sehingga cocok dengan perilaku API.

Pencatatan perilaku apa adanya tetap dipertahankan seperti pada Acara 21. Id dan kolom hasil PDO pada konfigurasi XAMPP dikirim sebagai string (misalnya "1"), sehingga pada skema Mahasiswa kolom id bertipe string dan contoh response memakai nilai string. Deskripsi operasi PUT dan DELETE menyebut bahwa keberadaan id tidak diperiksa dan bahwa PUT dengan body {} menghasilkan 500 tanpa JSON. Tombol try-it-out pada Swagger Editor mengirim request langsung dari browser, sehingga memerlukan header CORS pada server; API ini belum mengirim header tersebut, sehingga pengujian endpoint tetap dilakukan melalui Postman.

KESIMPULAN

1. Berkas docs/openapi.yaml berhasil dibuat dan memuat path /register, /login, /mahasiswa (GET, POST, PUT, DELETE), dan /buku dengan path yang mengikuti peta Router.

2. Komponen securitySchemes bertipe http/bearer (bearerAuth) ditambahkan, dan operasi yang dilindungi token ditandai dengan security bearerAuth.

3. Dokumen berhasil divalidasi oleh openapi-spec-validator dan Redocly CLI, lalu divisualisasikan pada Swagger Editor dengan tangkapan layar disimpan pada docs/.

4. Tugas mandiri selesai: path PUT dan DELETE /mahasiswa serta /register dilengkapi, dan securitySchemes ditambahkan pada components.

DAFTAR PUSTAKA

Pratama, R. A., & Hasanah, Q. (2026). Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Jurusan Teknologi Informasi, Politeknik Negeri Jember.